ТЗ / 01

TeamDocs AI

Техническое задание

Веб-приложение с RAG-чатботом
для работы с README-документацией
проектов команды

КОНЦЕПЦИЯ

Проект создаётся вручную README загружаются в интерфейсе
 система индексирует документы чат отвечает с источниками

Версия 1.0 · 05 ноября 2026 г.

Статус: проект документа для согласования

Содержание

Документ включает требования к продукту, проектную архитектуру, сценарии работы и концептуальные макеты веб-интерфейса.

1. Общие сведения . . . 3

2. Цель и состав проекта . . . 3

3. Функциональные требования . . . 3

4. Требования к веб-интерфейсу . . . 5

5. Архитектура и потоки данных . . . 6

6. Технологический стек . . . 7

7. Нефункциональные требования и безопасность . . . 8

8. Основные сценарии использования . . . 9

9. Критерии приёмки . . . 9

10. Этапы разработки и результаты . . . 10

11. Перспективы развития . . . 10

12. Вопросы для согласования . . . 10

Приложение А. Концептуальные макеты интерфейса . . . 11

Ключевое ограничение

GitHub, GitLab и другие Git-сервисы не подключаются. Проекты создаются, а README-файлы загружаются вручную через веб-интерфейс.

1. Общие сведения

Наименование: TeamDocs AI — веб-приложение с RAG-чатботом для работы с README-документацией проектов команды.

Назначение: централизованное хранение текстовой документации и интеллектуальный поиск по ней с возможностью получать ответы на естественном языке.

Целевая аудитория: разработчики, тестировщики, DevOps-инженеры, технические руководители и другие участники команды.

> **Принцип работы**
>
> Пользователь создаёт проект, загружает README, система индексирует файлы; чат ищет релевантные фрагменты и формирует ответ с указанием источников.

2. Цель и состав проекта

Разработать единое веб-приложение для размещения README-файлов по отдельным проектам и поиска информации в них с помощью технологии Retrieval-Augmented Generation (RAG).

В состав системы входят:

- каталог проектов с возможностью создания, редактирования и удаления;
- механизм ручной загрузки README и управления версиями документов;
- подсистема обработки Markdown, разбиения на фрагменты и построения эмбеддингов;
- векторная база знаний с привязкой фрагментов к проектам;
- AI-чат с поиском по одному, нескольким или всем доступным проектам;
- история диалогов, просмотр использованных источников и экран состояния индексации.

Вне объёма MVP: подключение к GitHub/GitLab, импорт через Git API, автоматическое чтение репозиториев и синхронизация изменений с Git.

3. Функциональные требования

3.1. Управление проектами

- Создание проекта с обязательным названием и необязательным описанием.
- Просмотр списка проектов, открытие карточки и редактирование атрибутов.
- Привязка к одному проекту произвольного количества поддерживаемых README-файлов.
- Удаление проекта и исключение всех его документов из поискового индекса.
- Логическое разграничение документации разных проектов; при многопользовательской работе — учёт прав доступа.

Пример: проект «Authentication Service» с файлами **README.md**, **README_SETUP.md** и **README_DEPLOY.md**.

3.2. Загрузка и управление документацией

- Загрузка одного или нескольких файлов через кнопку выбора или Drag & Drop.
- Поддержка форматов **.md**, **.markdown**, **.txt** в первой версии.
- Вывод списка файлов, размеров, дат загрузки и статусов обработки.

- Просмотр содержимого файла в веб-приложении.
- Замена файла новой версией с обновлением поискового индекса.
- Удаление документа и связанных с ним поисковых фрагментов.
- Проверка допустимых форматов, ограничений размера, пустых и повторно загруженных файлов.

Статусы: **«Ожидает обработки»**, **«Обрабатывается»**, **«Готов к поиску»**, **«Ошибка обработки»**.

3.3. Обработка и индексация README

После загрузки система запускает обработку в фоновом режиме, не блокируя интерфейс пользователя.

- Извлечение текста и структурных элементов Markdown: заголовков, абзацев, списков, таблиц и блоков команд.
- Разбиение на смысловые фрагменты (chunks) с сохранением контекста раздела.
- Формирование эмбеддингов для каждого фрагмента.
- Сохранение текста, векторов и метаданных в БД.
- Привязка фрагмента к проекту, документу, версии и заголовку раздела.
- Исключение устаревших фрагментов после замены или удаления файла.
- Отображение результатов обработки и ошибок в интерфейсе.

Предварительные параметры chunking: **400–800 токенов** с небольшим перекрытием; окончательные значения определяются после тестирования качества поиска.

3.4. RAG-чатбот

Веб-чат должен поддерживать вопросы на русском языке с англоязычной технической терминологией, последовательные уточнения и отображение ответа по мере генерации.

Режим	Описание
Один проект	Поиск только по README выбранного проекта.
Все проекты	Поиск по документации всех проектов, доступных пользователю.
Несколько проектов	Поиск по выбранному набору проектов и сравнение содержащихся в них сведений.

- Формирование ответа на основе найденных релевантных фрагментов, а не произвольного предположения модели.
- Сохранение контекста предыдущих сообщений для уточняющих вопросов.
- Поддержка Markdown, списков, таблиц и блоков команд в ответах.
- Предоставление ссылок на фактически использованные документы и разделы.
- Сообщение об отсутствии подтверждающих данных, если в документации нет ответа.
- Проверка разрешённой области поиска при каждом запросе.

3.5. Источники и проверяемость ответов

Каждый ответ, основанный на документации, должен содержать блок «Источники» с названием проекта, файла и раздела. Нажатие открывает документ во встроенном просмотрщике с выделением релевантного фрагмента.

Пример: «Authentication Service README_SETUP.md Переменные окружения».

3.6. История диалогов

- Создание нового диалога и сохранение сообщений пользователя и ассистента.
- Просмотр списка диалогов и продолжение выбранного диалога.
- Сохранение области поиска и метаданных использованных источников.
- Удаление диалога по запросу пользователя.

4. Требования к веб-интерфейсу

Интерфейс должен быть ориентирован на настольные устройства и адаптироваться к меньшим экранам. Основные действия выполняются без перехода на сторонние сервисы.

Экран	Основные элементы
Главная страница	Карточки проектов, поиск по названиям, кнопка создания проекта, переход к общему AI-чату, последние диалоги.
Страница проекта	Название и описание, Drag & Drop, список файлов, статусы обработки, просмотр, замена, удаление, кнопка чата по проекту.
AI-чат	Левая панель проектов и истории, центральный диалог, выбор области поиска, правая панель источников.
Настройки	Параметры индексации и поиска, выбранная модель, состояние обработки, параметры прав доступа.

В интерфейсе необходимо предусмотреть состояния загрузки, пустых списков, ошибок обработки, отсутствия результатов и недоступности языковой модели.

Концептуальные макеты приведены в приложении А. Они описывают компоновку и состав экранов, а не готовый интерактивный дизайн.

5. Архитектура и потоки данных

5.1. Пользовательский процесс

ОСНОВНОЙ ПОЛЬЗОВАТЕЛЬСКИЙ ПРОЦЕСС



Рисунок 1 — Последовательность действий от создания проекта до ответа чатбота.

5.2. Логическая архитектура

ЛОГИЧЕСКАЯ АРХИТЕКТУРА

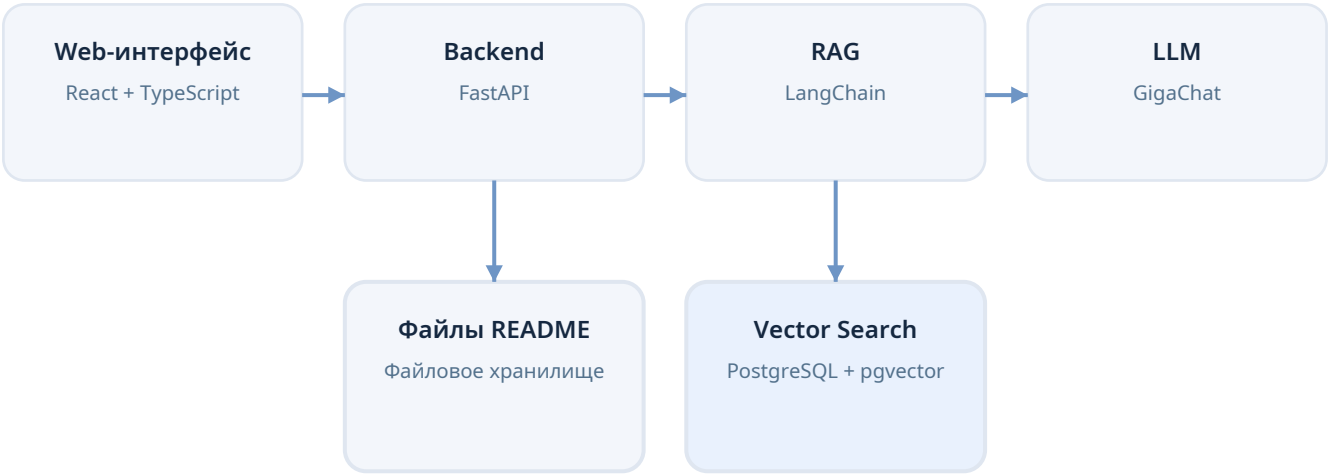


Рисунок 2 — Упрощённая архитектура веб-приложения. Внешние Git-интеграции отсутствуют.

5.3. Алгоритм загрузки и индексации

- Пользователь выбирает проект и загружает README через веб-интерфейс.
- Backend сохраняет исходный файл и создаёт задачу фоновой обработки.
- Обработчик извлекает Markdown, выделяет фрагменты и связывает их с разделами.
- Сервис эмбеддингов строит векторные представления, которые сохраняются в pgvector.
- После завершения обработки документ получает статус «Готов к поиску».
- При замене или удалении документа старые фрагменты и эмбеддинги исключаются из поиска.

5.4. Алгоритм ответа на вопрос

- Пользователь задаёт вопрос и выбирает проекты, по которым разрешён поиск.
- Backend проверяет область доступа и передаёт запрос в RAG-конвейер.
- Выполняются векторный и при необходимости полнотекстовый поиск и ранжирование фрагментов.
- Найденный контекст передаётся в языковую модель вместе с вопросом.
- Чат показывает ответ и ссылки на исходные документы внутри приложения.

5.5. Сущности хранения данных

Сущность	Ключевые поля
Проект	ID, название, описание, дата создания, владелец/группа доступа.
Документ	ID, ID проекта, имя файла, путь к файлу, версия, дата загрузки, статус обработки.
Фрагмент	ID, ID документа, текст, заголовок раздела, позиция, вектор эмбеддинга.
Диалог	ID, пользователь, выбранные проекты, дата создания и обновления.
Сообщение	ID, ID диалога, роль, текст, время, ссылки на источники.

6. Технологический стек

Для MVP предлагается следующий стек. Конкретные версии библиотек и модели фиксируются на этапе начала реализации.

Слой	Рекомендуемая технология
Frontend	React, TypeScript, Tailwind CSS
Backend	Python, FastAPI
RAG-конвейер	LangChain
Сложная оркестрация	LangGraph — опционально, вне обязательного MVP
Языковая модель	GigaChat
Эмбеддинги	GigaChat Embeddings
Реляционные данные	PostgreSQL
Векторный поиск	pgvector
Фоновые задачи	Celery, Redis
Хранение файлов	Локальное хранилище или MinIO

Слой	Рекомендуемая технология
Развёртывание	Docker Compose

> **Разграничение API**
>
> Интеграции с Git API отсутствуют. FastAPI используется как сервер веб-приложения, а GigaChat API — для генерации ответов и эмбеддингов.

На начальном этапе рекомендуется реализовать классический RAG без агентной маршрутизации. LangGraph имеет смысл добавлять при появлении многошаговых сценариев.

7. Нефункциональные требования и безопасность

7.1. Требования к работе приложения

- Корректная работа в актуальных версиях современных браузеров.
- Адаптивное отображение на настольных и мобильных устройствах.
- Асинхронная обработка файлов без блокировки пользовательского интерфейса.
- Индикация загрузки, прогресса индексации и ошибок.
- Понятное уведомление при сбоях индексации, БД или языковой модели.
- Возможность резервного копирования файлов и базы данных.
- Журналирование системных ошибок и операций обработки без раскрытия чувствительных данных.

7.2. Требования к безопасности

- Проверка загрузок: разрешённые расширения, размер, содержимое и безопасное имя файла.
- Запрет исполнения команд, скриптов или инструкций, находящихся внутри README.
- Снижение риска prompt injection: текст документа считается данными, а не системными инструкциями.
- Хранение секретов и ключей языковой модели на серверной стороне.
- Проверка прав доступа к проектам при загрузке документов, просмотре источников и выполнении RAG-запросов.
- Защита передачи данных; политика хранения истории чатов и резервных копий.
- Предварительное согласование возможности передачи содержимого внутренней документации поставщику LLM.

7.3. Качество поиска и ответов

Для приёмки необходимо подготовить набор контрольных вопросов с эталонными ответами и точными ссылками на исходные разделы README. Отдельно проверяются вопросы без ответа в документации и попытки получить сведения из недоступных проектов.

Числовые показатели качества, производительности, лимиты загрузки и условия хранения диалогов фиксируются после согласования объёма документации и инфраструктуры.

8. Основные сценарии использования

Сценарий 1. Создание базы знаний

Пользователь открывает приложение, создаёт проект, указывает название и описание, загружает несколько README. После индексации статус файлов меняется на «Готов к поиску».

Результат: новый проект доступен для поиска в AI-чате.

Сценарий 2. Вопрос по конкретному проекту

Пользователь открывает проект «Authentication Service» и спрашивает: «Как запустить сервис локально?» Чат ищет фрагменты только в документах этого проекта.

Результат: ответ с инструкцией и ссылками на конкретные README.

Сценарий 3. Поиск по всем проектам

Пользователь выбирает «Все проекты» и задаёт вопрос: «В каких сервисах используется Redis?»

Результат: сводный ответ с указанием проектов, где Redis подтверждён документацией.

Сценарий 4. Сравнение проектов

Пользователь выбирает два проекта и спрашивает: «Чем различаются инструкции по развёртыванию?»

Результат: сравнение, опирающееся на README выбранных проектов.

Сценарий 5. Замена README

Пользователь загружает обновлённую версию README.md. Система пересоздаёт связанные поисковые фрагменты.

Результат: новые ответы используют актуальный текст документа.

Сценарий 6. Отсутствие ответа

Пользователь спрашивает о параметре, который не описан в загруженной документации.

Результат: чат сообщает об отсутствии подтверждённых сведений без выдумывания значения.

9. Критерии приёмки

Приложение может быть принято после демонстрации следующих сценариев на тестовых README-файлах:

01. Создание и редактирование проекта через веб-интерфейс.

02. Загрузка нескольких README-файлов в один проект.

03. Автоматическая индексация после загрузки с корректным отображением статуса.

04. Просмотр, замена и удаление загруженного документа.

05. Чат по одному проекту, нескольким выбранным проектам и всем доступным проектам.

06. Ответы с указанием README и релевантных разделов.

07. Открытие источника непосредственно из ответа в приложении.

08. Корректная реакция на отсутствие ответа в документации.

09. Исключение устаревшего текста из индекса после замены README.

10. Исключение документа из поиска после его удаления.

11. Сохранение, открытие и удаление истории диалогов.

12. Соблюдение прав доступа к проектам при многопользовательской работе.

13. Документированные процедуры запуска, настройки и резервного копирования.

10. Этапы разработки и результаты

Этап	Содержание	Результат
1. Backend и хранение	Модели данных; CRUD проектов; загрузка и хранение документов; фоновые задачи.	Создание проектов и загрузка файлов.
2. RAG	Markdown-парсер; chunking; embeddings; pgvector; LangChain; GigaChat; источники.	Поиск и генерация ответов.
3. Веб-интерфейс	Каталог проектов; страница проекта; загрузка README; чат; история; просмотр источников.	Рабочий пользовательский интерфейс.
4. Тестирование и внедрение	Проверка индексации, удаления, безопасности, релевантности; подготовка документации.	Версия для приёмки и эксплуатации.

Итоговые материалы: работающее веб-приложение, серверная часть, механизм индексации, база знаний, конфигурация развёртывания, руководство пользователя, инструкция администратора и протокол тестирования.

11. Перспективы развития

- Редактирование README непосредственно в браузере.
- Поддержка PDF, DOCX и других форматов документов.
- Автоматическое составление описаний проектов и FAQ.
- Сравнение документации и поиск противоречий между проектами.
- Совместная работа с проектами и расширенные роли.
- Аналитика запросов и выявление недостаточно документированных тем.
- Усложнение RAG-конвейера с помощью LangGraph при необходимости.

12. Вопросы для согласования

- Будет ли система доступна одному пользователю или нескольким участникам команды?
- Как разграничиваются права к проектам и их документации?
- Каков ожидаемый объём файлов и максимально допустимый размер одного README?
- Где разворачивается приложение и где сохраняются загруженные файлы?
- Допускается ли передача внутренней документации во внешний GigaChat API?
- Какой срок хранения истории диалогов и резервных копий требуется?

Приложение А. Концептуальные макеты интерфейса

Эскизы иллюстрируют состав и расположение элементов. Они не являются готовым дизайном или программным кодом.

А.1. Страница проекта и загрузка README

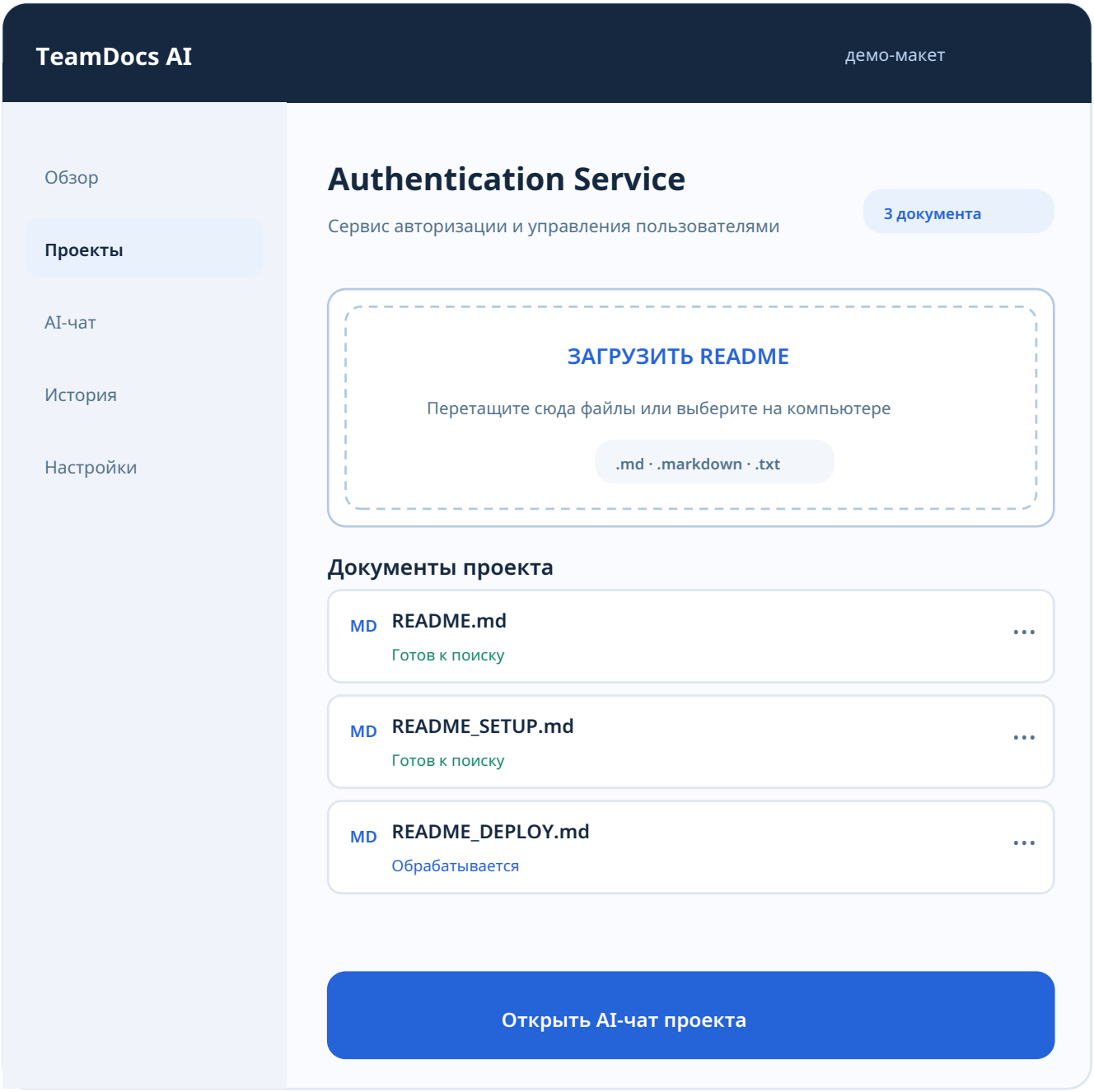


Рисунок А.1 — Карточка проекта с областью Drag & Drop, списком загруженных файлов, статусами и переходом в AI-чат.

A.2. Страница AI-чата

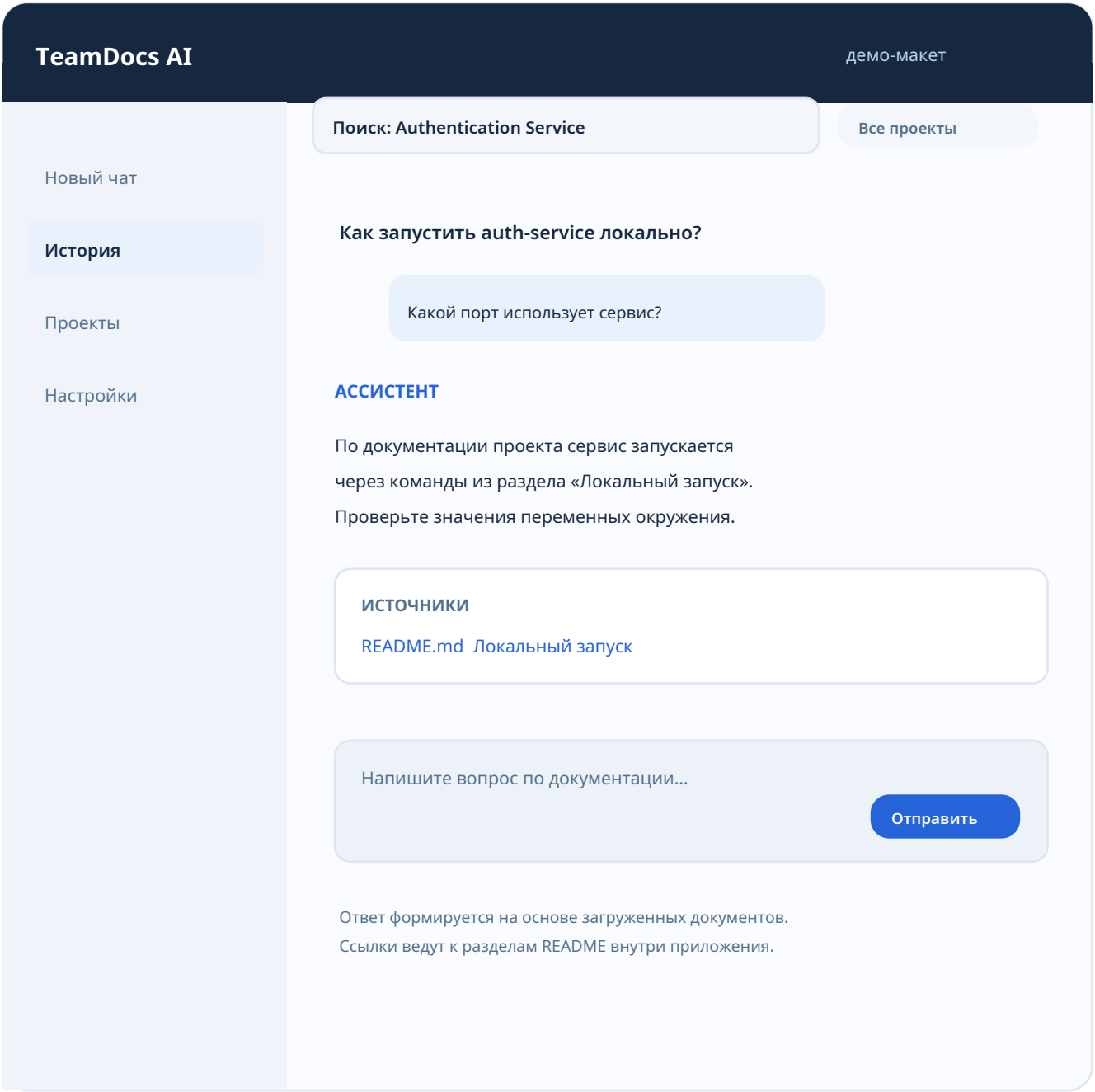


Рисунок А.2 — Пример диалога с выбором области поиска, ответом ассистента и ссылкой на исходный README.

Для полного интерфейса рекомендована правая панель источников. В компактном представлении источники показаны непосредственно под ответом.

А.3. Управление базой знаний

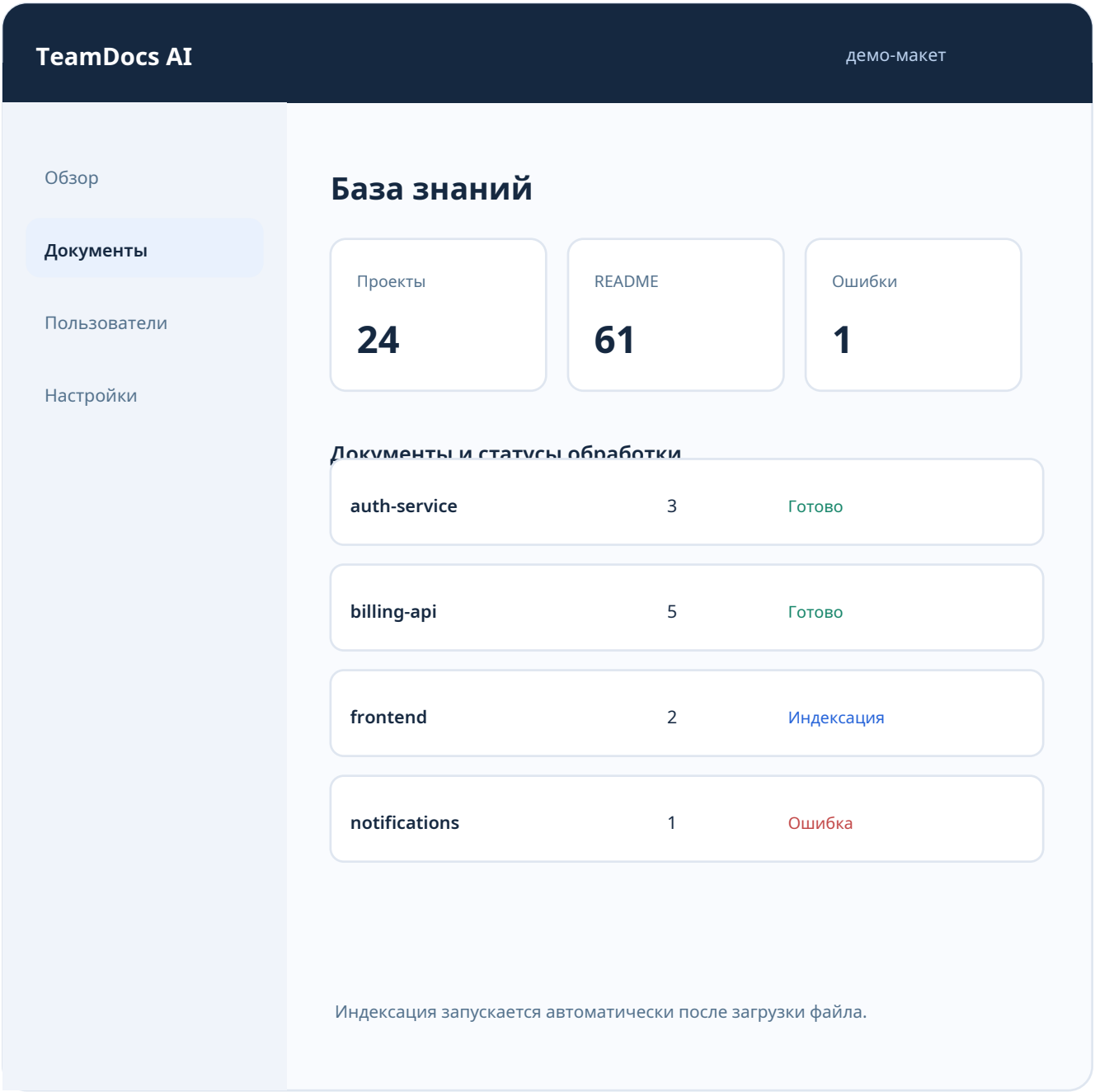


Рисунок А.3 — Административный экран со сводными показателями и состоянием обработки документов. Числа и названия демонстрационные.

Замечание: данные в макетах условные и не отражают реальные проекты команды.